설비 고장을 예측하는 AI 데이터 분석

Pandas 기초

DataFrame 구조와 통계 미리보기

posco

CONTENTS

목차

01 데이터 미리보기와 구조 파악

02 통계 미리보기와 이상 신호 읽기

03 첫 탐색 리포트 만들기

posco

posco

01

데이터 미리보기와 구조 파악

head · tail · shape · columns · index · dtypes · info

posco

탐색 단계 — 무엇을 보나

불러온 다음 바로 분석으로 가지 않고 눈으로 먼저 살펴보는 일 — 이것을 탐색이라 부름

01 · 크기

몇 줄, 몇 열짜리 데이터인가

요리로 치면 재료가 얼마나 왔는지 펼쳐 보는 일

02 · 내용

실제 값은 어떻게 생겼는가

열 이름과 값을 직접 눈으로 확인

03 · 이상

빈 칸이나 이상한 값은 없는가

결측·이상값을 미리 발견해야 분석 오류 예방

탐색을 건너뛰면 잘못된 데이터로 분석하게 됨 — 첫 도구는 가장 자주 쓰는 head

posco

head() — 앞부분 미리보기

데이터를 불러오면 가장 먼저 치는 확인 명령 — 앞에서 5줄 보기

CODE

```
import pandas as pd
# CSV를 표로 불러오기
df = pd.read_csv('data/12_metro_compressor.csv')
# 앞 5줄 (일반 .py 파일에서는 print 필요)
print(df.head())
```

열 이름·값·빈 칸을 한눈에 확인

posco

head로 무엇을 보나

앞 5줄만 봐도 결측 신호까지 함께 잡힘

CHECK 01

열 이름이 제대로 나왔는지, 값이 칸에 맞는지 확인

맨 위 열 이름이 의도대로 표시되고, 각 칸에 들어간 값이 어긋나지 않았는지

CHECK 02

빈 칸(NaN)이나 이상한 값이 없는지 빠르게 점검

압축기 데이터는 인덱스 3번 행 오일온도가 NaN — 앞 5줄만 봐도 결측이 바로 눈에 띔

posco

tail() — 뒷부분 미리보기

데이터가 끝까지 제대로 쌓였는지 확인 — 뒤에서 5줄 보기

CODE

```
# 뒤에서 5줄 — 기본
df.tail()

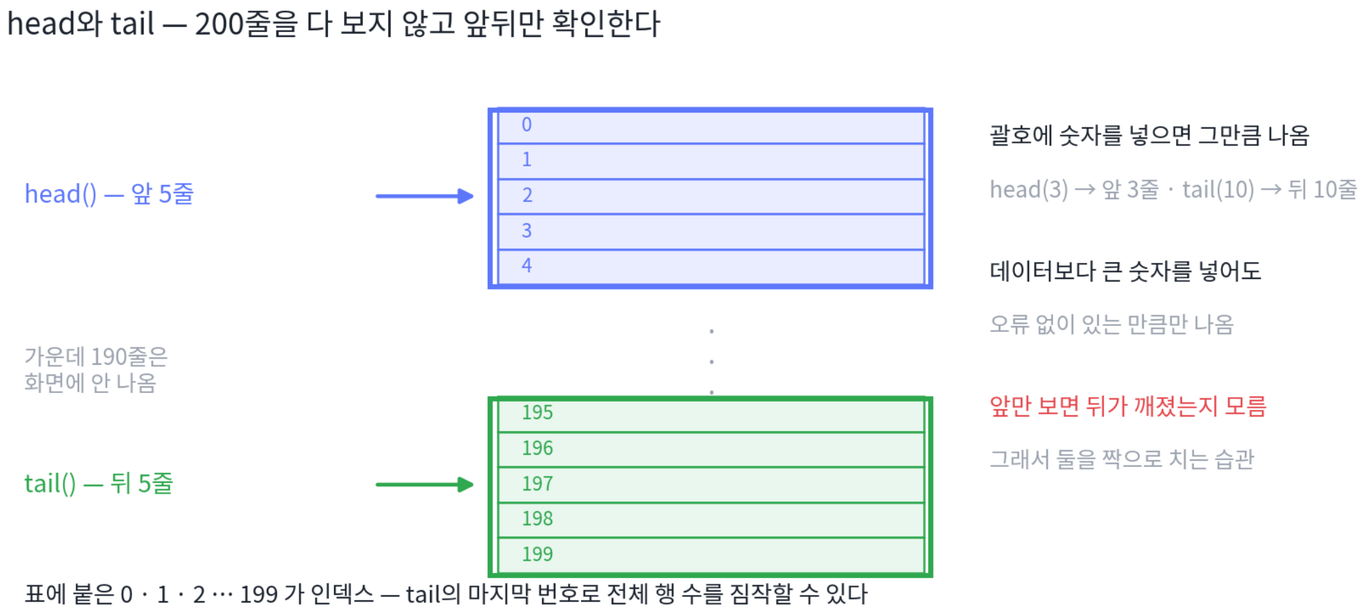
# 뒤에서 3줄만 — 숫자 지정
df.tail(3)

# → 파일 끝 깨짐·엉뚱한 줄 점검에 필수
```

head로 시작과 tail로 끝을 살펴보되, 가운데 데이터 품질은 별도 검사해야 함

head와 tail이 보여 주는 자리

200줄을 다 보지 않고 앞뒤 몇 줄만으로 첫인상을 잡는다



head는 앞 5줄, tail은 뒤 5줄 — 괄호 안 숫자(인자)를 바꾸면 그만큼 나옴

데이터보다 큰 숫자를 넣어도 오류 없이 있는 만큼만 나옴 · tail의 마지막 번호 199로 전체 200줄을 짐작

앞만 보면 뒤가 깨졌는지 모름 — 그래서 둘을 짝으로 치는 습관이 필요함

posco

미리보기에서 만나는 NaN

앞 단원에서 찾아 헤매던 빈 자리가 표에서는 눈에 바로 보인다

미리보기에서 NaN을 만나면 — 오류가 아니라 값이 비었다는 표시

	측정시각	압축압력	오일온도	모터전류
0	06:38:47	9.30	51.3	6.04
1	07:28:21	8.55	56.8	0.04
2	08:17:54	8.67	55.7	0.03
3	09:07:27	9.76	NaN	3.81
4	09:57:01	8.49	55.3	0.04

값이 0인 것과 다르다

0은 측정해서 0이 나온 것,

NaN = Not a Number

센서가 그 시각 값을 못 남긴 자리

지금 단계의 목표는 발견까지 — 채울지 지울지는 뒤 단원에서 다룬다

앞 단원에서는 빈 자리를 찾는 명령을 따로 쳐야 보였지만, 표에서는 NaN으로 그냥 보임

숫자 0과 전혀 다름 — 0은 측정해서 0이 나온 것, NaN은 측정 자체가 없는 것

지금 단계의 목표는 발견까지 — 채울지 지울지는 뒤 단원에서 다룸

posco

실습 1. 앞뒤를 미리 보고 첫인상 잡기

압축기 데이터의 앞뒤를 확인하고 보이는 줄 수를 바꿔 보기

목표

앞뒤 몇 줄만 보고 전체 크기와 빈 칸을 짐작해 보기

단계

① 압축기 파일을 불러와 변수에 담기

② 앞부분을 기본 줄 수로 화면에 띄우기

③ 뒷부분을 띄워 마지막 번호가 몇인지 확인하기

④ 괄호 안 숫자를 바꿔 앞 세 줄과 뒤 열 줄 보기

⑤ 앞부분에서 빈 칸으로 보이는 자리를 찾아 적어 두기

힌트 | df.head() / df.tail() · head(3) / tail(10) · 마지막 인덱스 199

예상 결과

마지막 번호 199 · 앞 5줄 중 네 번째 줄 오일온도가 비어 있음

posco

.shape — 행·열 크기

괄호 없는 속성 — 앞이 행, 뒤가 열

CODE

```
df.shape

# (200, 7) → 200행, 7열

# head와 달리 괄호를 안 붙임 – 속성이라서
# 행 먼저, 열 나중 – Pandas 전체에서 같은 순서
```

결과 (200, 7)에서 앞이 행 뒤가 열 · 속성이라 괄호를 붙이면 오류 — 다음은 열 이름

posco

.columns — 열 이름 목록

데이터를 골라내는 열쇠 — 정확한 이름 확인

CODE

```
df.columns

# ['측정시각', '압축압력', '배출압력', '저장압력',
#  '오일온도', '모터전류', '가동상태']

# 모든 열 이름을 목록으로 보여주는 속성
```

데이터를 골라내려면 정확한 철자를 확인해야 함

posco

columns — 왜 중요한가

철자 한 글자만 달라도 못 찾음 — 정확한 이름이 곧 열쇠

POINT 01

'오일온도' 열을 보려면 정확히 '오일온도'라고 써야 함

'오일 온도'처럼 띄어쓰기가 하나 다르거나 뒤에 공백이 붙어도 못 찾음

POINT 02

열이 수십 개인 파일에서 전체 목록을 확인할 때 특히 유용

head로는 열이 많으면 가운데가 생략되지만 columns는 모든 열 이름을 표시

열 이름을 봤으니 행에 붙은 이름표, index로

posco

.index — 행 번호표

괄호 없는 속성 — 각 행에 붙은 이름표를 목록으로

CODE

```
df.index

# RangeIndex(start=0, stop=200, step=1)
# 0부터 199까지 1씩 — 파일에 없던 번호를 read_csv가 붙여 준 것

len(df.index)     # 200 — shape의 앞 숫자와 같은 값
```

행 번호표를 봤으니 이번엔 열마다의 자료형, dtypes로

posco

.dtypes — 열별 자료형

int64 정수 · float64 소수 · object/str 글자 — 이 세 가지면 자료형의 90% 해결

CODE

```
df.dtypes

# 오일온도   float64   소수점 있는 숫자
# 모터전류   float64   소수점 있는 숫자
# 가동상태   object    글자 (버전에 따라 str로 표시)

# 디지털 신호 파일은 0과 1뿐이라 정수로 읽힘
pd.read_csv('data/12_metro_digital.csv').dtypes
# 압축기     int64     소수점 없는 정수
```

자료형이 왜 중요한지, 잘못되면 어떤 문제가 생기는지로

posco

자료형이 분석에 미치는 영향 — 문제 상황

자료형이 틀리면 계산이 아예 안 되는 경우 발생

Q1 오일온도가 글자로 읽히면?

A 평균을 못 구함 — 글자끼리는 계산 불가

오일온도 열에 '-' 나 '측정불가' 같은 글자가 섞이면 Pandas가 그 열을 글자로 읽음

Q2 ID가 숫자로 읽히면?

A '001'의 앞자리 0이 사라져 '1'로 망가짐

우편번호·제품 코드도 같은 문제 발생

posco

읽을 때 자료형을 지정해 막기

숫자로 읽히면 안 되는 열은 처음부터 글자로 지정

CODE

```
# 상태 코드 0·1 은 계산할 값이 아니라 종류를 나타내는 기호
d = pd.read_csv('data/12_metro_digital.csv',
                dtype={'압축기': str})

print(d['압축기'].dtype)     # object (pandas 3부터는 str)

# 지정하지 않으면 정수로 읽힘
e = pd.read_csv('data/12_metro_digital.csv')
print(e['압축기'].dtype)     # int64

# 주의) 001 같은 코드는 숫자로 읽히면 1 이 되어 되돌릴 수 없음
#       설비번호 · 우편번호 · 제품코드는 처음부터 글자로 지정
```

posco

구조를 보는 네 도구가 표의 어디를 가리키는가

shape · columns · index · dtypes 를 한 그림에 겹쳐 놓기

구조를 보는 네 도구가 표의 어디를 가리키는가



네 도구를 합쳐 한 번에 보여 주는 것이 .info() — 거기에 결측 개수까지 붙는다

shape · columns · index · dtypes 는 괄호가 없는 속성이고, info() 는 괄호가 붙는 함수

columns는 맨 윗줄의 열 이름, index는 맨 왼쪽의 행 번호표

shape는 표 전체의 (행 수, 열 수), dtypes는 열마다의 자료형

네 도구를 합쳐 한 번에 보여 주는 것이 info — 거기에 결측 개수까지 붙는다

posco

실습 2. 표의 뼈대 읽기

크기 · 열 이름 · 자료형을 한 번씩 확인

목표

세 가지를 차례로 찍어 보고 이 표가 어떤 표인지 한 문장으로 정리

단계

① 디지털 신호 파일을 불러와 크기를 출력하고 몇 줄 몇 칸인지 적기

② 열 이름 목록을 출력해 그대로 옮겨 적기

③ 열마다의 자료형을 출력해 숫자 열과 글자 열로 나누기

④ 앞부분을 화면에 띄워 값이 몇 가지나 되는지 눈으로 보기

⑤ 「이 표는 ~이다」 형태로 한 문장 정리하기

힌트 | 파일: 12_metro_digital.csv · shape / columns.tolist() / dtypes / head()

예상 결과

120행 4열 · 숫자 열 세 개, 글자 열 한 개 · 값은 0과 1뿐

posco

.info() — 전체 구조 한눈에

shape · columns · dtypes · 결측을 종합한 만능 도구

CODE

```
df.info()

# RangeIndex: 200 entries
# 오일온도 199 non-null float64
# 모터전류 200 non-null float64

# shape와 달리 괄호 필요
```

head와 info만 쳐도 데이터의 절반은 파악한 셈

posco

info() 출력 읽기 — 행 수와 열

출력의 앞부분 — 행 수·열 수·열 이름 읽기

01 · 행 수

`RangeIndex: 200 entries`

→ 200행

맨 위 entries가 행 개수 — shape의 앞 숫자와 동일

02 · 열 수

`total 7 columns`

→ 7열

shape의 뒤 숫자와 동일한 값

03 · 열 이름

`Column` 칸

→ 세로 나열

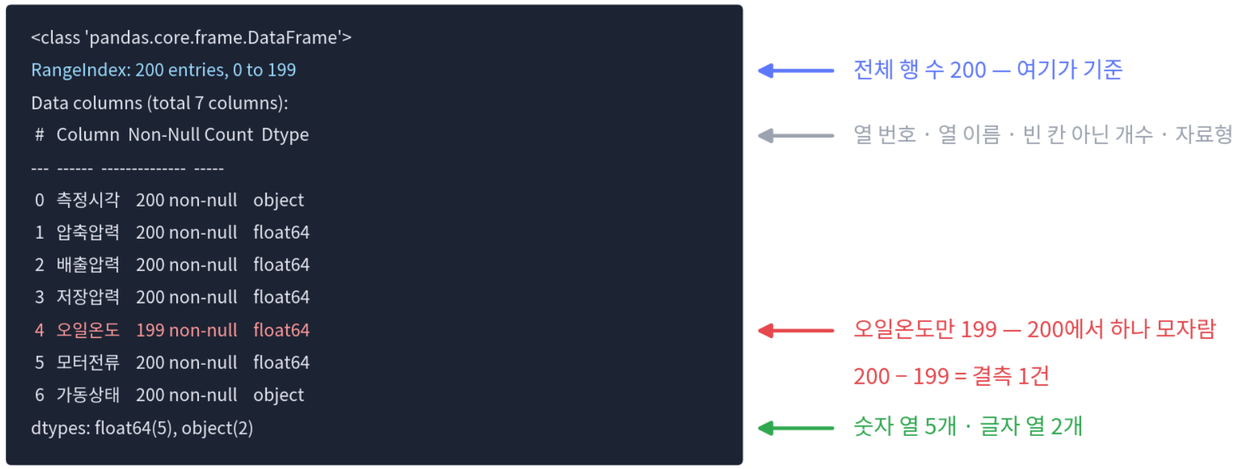
측정시각·압축압력 같은 열 이름이 세로로 표시

posco

info 출력에서 무엇을 읽는가

Non-Null Count 한 칸이 결측을 정확한 개수로 알려 준다

info() 출력에서 무엇을 읽는가 — Non-Null Count 한 칸이 핵심



head는 앞 5줄에서 우연히 보이는 결측만 잡지만, info는 전체에서 정확한 개수로 알려 준다

맨 위 entries가 전체 행 수 — 이 숫자가 결측을 세는 기준이 됨

오일온도만 199 — 200에서 빼면 결측이 1건이라는 계산이 나옴

오른쪽 Dtype 칸에는 열마다의 자료형이 함께 표시 — 결측과 자료형을 한 화면에서 확인

posco

실습 3. info로 결측까지 한 번에 보기

한 줄로 크기 · 자료형 · 결측을 동시에 확인하기

목표

info 출력에서 결측이 있는 열을 찾아 개수를 계산해 보기

단계

① data/12_metro_compressor.csv를 불러온 뒤 df.info() 실행

② 맨 위 줄에서 전체 행 수를 찾아 적기

③ 빈 칸 아닌 값의 개수가 전체보다 작은 열 찾기

④ 전체 행 수에서 그 값을 빼 결측 개수 계산하기

힌트 | 파일: 12_metro_compressor.csv · info() · 200 - 199 = 결측 1건

예상 결과

전체 200행 · 오일온도만 199 → 결측 1건

posco

02

통계 미리보기와 이상 신호 읽기

describe · 평균과 중앙값 · 이상 신호 · 두 파일 비교

posco

.describe() — 기초 통계 요약

숫자 열들의 평균·최대·최소를 한 번에 자동 계산

CODE

```
import pandas as pd
df = pd.read_csv('data/12_metro_compressor.csv')
print(df.describe().round(2))
# 오일온도 평균 63.18, 최대 75.0
# 숫자 열 5개를 한꺼번에 요약
```

앞 단원에서 하나씩 구하던 통계를 숫자 열 전부에 대해 한 번에 계산

posco

앞 단원에서 하나씩 구하던 것이 한 줄로

같은 통계를 훨씬 적은 코드로 — 그리고 더 파고들 때는 배열로 돌아가면 됨

CODE

```
import numpy as np
# 앞 단원: np.mean(x) / np.median(x) 등 따로 계산
# Pandas: 숫자 열 전체 요약을 한 번에
print(df.describe())
# 조건별 비교는 앞에서 배운 NumPy 방식도 활용
state = df['가동상태'].values
cur = df['모터전류'].values
print(round(float(np.mean(cur[state == '가동'])), 2))
# 출력: 4.25
```

describe로 전체를 훑고, 수상한 열만 배열로 꺼내 파고드는 것이 실무 순서

posco

통계로 이상 신호 감 잡기

앞 단원에서 사분위수로 하던 판단을 describe 표 한 장에서 바로

열	75%	최댓값	이상의심
오일온도	68.1	75.0	추가 확인
배출압력	-0.02	0.60	이상 후보
모터전류	3.81	6.19	추가 확인

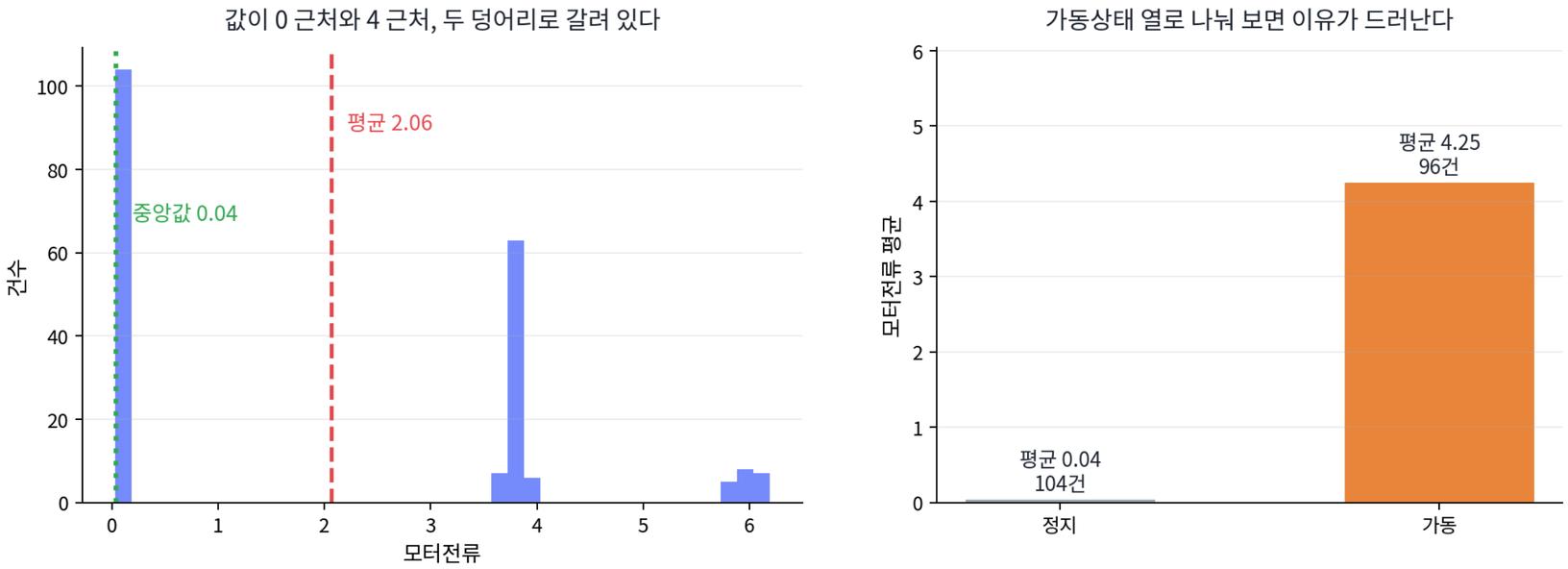
> 배출압력은 분포에서 벗어난 값 후보가 있음 — 실제 이상 여부는 설비 기준·운전 맥락 확인

수상한 열을 찾았으면 그 열만 배열로 꺼내 파고들면 됨

평균이 한가운데 값의 50배가 되는 이유

서로 다른 두 상태가 한 열에 섞여 있으면 평균이 실제를 대표하지 못한다

평균이 중앙값의 50배 — 서로 다른 두 상태가 한 열에 섞여 있다는 신호



모터전류는 0 근처와 4 근처, 두 덩어리로 갈려 있음 — 그 사이 값은 거의 없음

가동상태로 나누면 가동 4.25 · 정지 0.04 — 두 상태를 섞어 평균 낸 값이 2.06

평균 하나만 보고 판단하면 실제로는 존재하지 않는 상태를 대표값으로 삼게 됨

posco

실습 4. 통계 요약에서 이상 신호 찾기

평균과 한가운데 값이 어긋난 열을 찾아 이유 설명하기

목표

숫자만 보고 수상한 열을 찾아낸 뒤 그 이유를 데이터로 확인하기

단계

① 압축기 파일에 통계 요약을 적용해 출력하기

② 열마다 평균과 한가운데 값을 나란히 적기

③ 두 값이 크게 어긋난 열을 찾기

④ 그 열과 가동상태 열을 각각 배열로 꺼내기

⑤ 가동일 때와 정지일 때의 평균을 따로 구해 비교하기

힌트 | describe()의 mean·50% 비교 → 가동상태별 모터전류 평균

예상 결과

모터전류 평균 2.06 대 한가운데 값 0.04 — 가동 4.25 · 정지 0.04

posco

두 데이터 비교 — 교훈

같은 설비에서 나왔는데 성격이 전혀 다름 — 같은 도구도 읽는 법이 달라짐

CONTINUOUS

연속으로 변하는 값

압축기 파일

압축기 파일 — 소수점 실수, 200행 7열, 오일온도에 결측 1건

DIGITAL

켜짐과 꺼짐만 있는 신호

디지털 파일

디지털 파일 — 0 또는 1인 정수, 120행 4열, 결측은 0건

CAUTION

주의

결측이 없다고 좋은 것은 아님

저압스위치는 이 120줄에서 모두 0 — 이 구간의 상태 변화 판단에는 정보가 적음

같은 순서로 두 파일을 훑어 무엇이 다른지 직접 확인해 보기

posco

실습 5. 성격이 다른 두 파일 비교하기

같은 순서로 두 파일을 훑어 무엇이 다른지 정리

목표

같은 도구를 써도 파일 성격에 따라 읽는 법이 달라짐을 확인

단계

① 압축기 파일과 디지털 파일을 각각 다른 이름으로 담기

② 두 파일의 크기를 나란히 출력해 비교하기

③ 두 파일의 자료형을 비교해 숫자의 종류가 어떻게 다른지 보기

④ 각 파일에 구조 요약을 적용해 빈 칸 아닌 값의 개수를 비교하기

⑤ 값이 한두 가지뿐인 열이 있는지 배열로 꺼내 확인하기

힌트 | 두 CSV 각각 읽기 · shape / dtypes / info / np.unique(..., return_counts=True)

예상 결과

200행 7열 대 120행 4열 · 결측 1건 대 0건 · 저압스위치는 120줄 내내 0

posco

데이터 첫 탐색 체크리스트

head → shape → info → describe, 네 단계의 흐름

STEP 1

head

실제 값 확인 — 무엇에 관한 데이터인가

큰 그림 파악

→

STEP 2

shape

크기 확인 — 얼마나 큰 작업인가

행·열 수

→

STEP 3

info

구조와 결측 점검

Non-Null

→

STEP 4

describe

통계와 이상 신호 — 첫 탐색의 마침표

리포트로

무엇인가 → 얼마나 큰가 → 어디가 비었나 → 값이 정상인가, 네 질문의 순서

03

탐색 리포트 만들기

지금까지 배운 네 단계를 순서대로 이어 한 장으로 정리하기

posco

실습 6. 탐색 리포트 만들기

네 단계를 순서대로 이어 다섯 줄짜리 리포트로 정리

목표

배운 도구를 순서대로 이어 처음 보는 파일 한 장 요약을 완성하기

단계

① 작은 파일(12_metro_small.csv)을 불러오고 head()로 내용 확인

② shape로 크기 확인, info()로 결측 있는 열 찾기

③ 30행에서 빈 칸이 아닌 값 수를 빼 결측 개수 계산

④ describe()의 평균(mean)·중앙값(50%)으로 특이 열 찾기

⑤ 숫자를 근거로 간단한 리포트와 다음 확인 사항 작성

힌트 | 파일: 12_metro_small.csv · head → shape → info → describe → 해석 문장

예상 결과

30행 7열 · 결측 1건 · 모터전류 평균 1.38 대 한가운데 값 0.04